

系列モデルを用いた大規模ファイル断片の種別識別

片野 凱介[†]

[†]有明工業高等専門学校 創造工学科 情報システムコース
Gauthier Lovic 研究室

Large-scale File Fragment Type Identification using Sequence Models

Yoshiyuki Katano[†]

[†] Information System Course, Department of Creative Engineering,
National Institute of Technology (KOSEN), Ariake College
Gauthier Lovic Laboratory

2026 年 2 月 25 日

Abstract– 本研究は、マジックナンバーなどのファイルヘッダー情報を含まない場合もある短いバイト列断片から、元のファイル種別を推定する「ファイル断片種別識別」を対象とする。先行研究 FiFTy は FFT-75 データセットを用い、CNN により 512 バイト断片 (512B scenario #1 75 クラス) で一定の識別性能を示しているが、バイト列を系列として扱う自然言語処理系モデル (RNN/Self-Attention) の有効性は十分に整理されていない。そこで本研究では、FFT-75 の 512B scenario #1 (訓練データ数 6,144,000/検証データ数 768,000/テストデータ数 768,000) を用い、入力を 0-255 の整数系列として埋め込み表現に変換し、LSTM, GRU, Transformer を同一の前処理・損失・分類ヘッド設計の下で学習させ、FiFTy の考え方に基づく CNN ベースラインと比較した。

実験の結果、test Top-1 accuracy は GRU 70.10%, Transformer 68.10%, LSTM 67.90%, CNN 66.90% となり、Top-3 accuracy は GRU 77.90%, LSTM 76.10%, Transformer 76.00%, CNN 75.10% となった。このことから、512 バイトという短い断片条件においても、系列モデルが CNN ベースラインを上回り得ること、特に GRU が本設定で有効であることを示した。さらに、混同行列等の追加分析を通じて、断片長制約下で識別が難しいクラス関係と誤分類傾向を整理し、運用上の留意点を考察した。なお、75 クラス分類の都合、図の一部は末尾ページに付録として記載した。

Keywords: ファイル種別識別, file type classification, ファイル断片識別, file fragment classification, FFT-75, デジタルフォレンジクス, ファイルカービング, 機械学習, GRU, LSTM, Transformer

目次

1 序論.....	1
2 関連研究.....	1
2.1 ファイル断片識別とファイルカービング.....	1
2.2 手工特徴量に基づくファイル断片分類.....	2
2.3 深層学習によるファイル断片分類.....	2
2.4 FFT-75 データセットの概要.....	2
3 問題設定とデータセット.....	3
3.1 問題設定（タスク定義）.....	3
3.2 データセット FFT-75 と scenario #1.....	3
3.3 断片生成仕様（512B, ランダム位置）.....	3
3.4 データ分割（train/val/test）.....	3
3.5 入力表現と前処理（0-255, Embedding, padding/mask）.....	3
4 提案手法.....	3
4.1 記法と入出力.....	4
4.2 共通設計（Embedding・正則化・初期化・分類ヘッド）.....	4
4.3 CNN ベースライン（FiFTy 派生 1D-CNN）.....	4
4.4 LSTM ベースモデル.....	5
4.5 GRU ベースモデル.....	5
4.6 Transformer ベースモデル.....	5
4.7 パラメータ規模の整合（比較の公平性）.....	6
5 実験設定.....	6
5.1 実験環境.....	6
5.2 実験手順.....	6
5.3 学習条件.....	6
5.4 検証・評価.....	6
5.4.1 テスト評価時の混同行列.....	7
6 実験結果.....	8
6.1 総合性能（検証データ・テストデータ）.....	8
7 考察.....	9
7.1 総合性能差と系列モデル適用の含意.....	9
7.2 誤分類傾向と 512B 制約.....	9
7.3 バイト列表現設計の妥当性と今後.....	10
7.4 実運用上の示唆と研究上の限界.....	10
8 結論.....	10
謝辞.....	11
参考文献.....	11
付録.....	11
A 実装済みの使用しなかった機能.....	11
B 実行環境および主要ソフトウェア.....	11
B.1 実行環境.....	12
B.2 使用ライブラリ.....	12
C 公開レポジトリ.....	12
D 混同行列から得られる各モデルのクラス別指標.....	12
E 混同行列を図示したグラフ.....	17

1 序論

近年、メモリフォレンジクスやファイルカービングに代表されるバイナリ解析の現場では、断片化したデータから有用情報を抽出する必要性が高まっている¹。これらの処理では、対象断片がどの種類のファイルに由来するかを識別する「ファイル種別識別」が前処理として重要である。しかし、実運用で観測されるデータは必ずしもファイルヘッダーを含まず、また断片長も小さい場合が多い。このため、マジックナンバー等のヘッダー依存手法は適用が困難となり、短いバイト列断片のみからファイル種別を識別する技術が求められる。これらを機械的に解決する研究として、ファイルシグネチャやヒューリスティック規則により、データ断片の構造や型を識別する方法が従来のファイルカービング手法 [1] であるが、これらは一定の復元には有効だった一方、大量の断片化データや複雑なファイル構造を扱う場合には限界がある。なお、ファイル種別という言葉に一般に置き換えると、ファイル形式や拡張子が最も近い。本論文においては、ファイル種別 (File type) とは、PDF や JPEG といった粒度のものを指す²。

この課題に対し、深層学習を用いたアプローチが提案されている。先行研究 FiFTy [2] は、メモリフォレンジクスおよびファイルカービングを想定し、ファイル断片からのファイル種別識別を目的とする現代的な手法を提示している。また、評価用データセットとして FFT-75 [3] が公開されており、複数シナリオに基づく比較が可能である。

ファイル断片からのファイル種別識別では、入力情報が限定されるため、クラス間の判別に必要な特徴が十分に含まれない場合がある。加えて、その種別によっては圧縮・暗号化・コンテナファイル化等の影響により、バイト列の局所統計が類似し、単純な特徴量では識別が難しい。なお、コンテナファイル化という言葉に一般に置き換えると、ファイルの埋め込みが最も近い。本論文においてコンテナファイル化とは、あるファイルをそれとは異なる形式のファイルに埋め込む、つまり、コンテナファイル (Container File) にすることを指す。例えば、PDF に JPEG を埋め込むことを指す。

先行研究 FiFTy は CNN を中心とした設計により、断片中の局所パターンを抽出して分類性能を達成している。一方で、バイト列は系列データであり、自然言語処理と同様に、系列としての依存関係 (位置関係や広い文脈) をモデル化する設計も考えられる。自然言語処理分野では、系列モ

デルとして LSTM, GRU, Transformer が広く用いられており [4], 長短さまざまな依存関係を表現できるとされる。しかし、FFT-75 の 512B, scenario #1 という統一条件下で、これら系列モデルがファイル断片分類に対してどの程度有効かを整理した比較は十分ではない。

以上を踏まえて、本研究の目的は、FFT-75 データセットのうち 512 バイト断片を対象とする scenario #1 において、系列モデル (LSTM, GRU, Transformer) を適用し、ファイル種別識別における CNN と比較した系列モデルの有効性を実験的に検証することである。具体的には、データ分割, 前処理, 学習条件, 評価指標を可能な限り統一し、系列モデル間の比較に加えて、CNN ベースライン (先行研究 FiFTy の考え方に基づく条件を置いた実装) との相対比較を行う。また、付随して、モデル間の性能差と、誤分類の傾向についても考察する。

本研究の貢献を以下に示す。

1. データセット FFT-75 [3] (512B, scenario #1) における系列モデル (LSTM, GRU, Transformer) を比較し、ファイル断片分類における系列モデルの適用可能性を示した
2. 0-255 のバイト値を入力とする際の embedding などの表現を整理し、モデル間で共有すべき設計点と差異が生じる設計点を明確化した
3. 混同行列等³を用いて誤分類の傾向を分析し、512 バイト断片という制約下で識別が難しいクラス関係を考察した

本論文の構成を述べる。2 章では、ファイル断片からのファイル種別識別に関する関連研究を整理し、FiFTy の位置づけおよび FFT-75 の概要を示す。3 章では、本研究で扱う問題設定とデータセット, scenario #1 の意味, データ分割および前処理手順を定義する。4 章では、提案する系列モデルとして LSTM, GRU, Transformer ならびに CNN ベースラインの構成を述べ、5 章にて、その実験環境と学習条件, 評価指標を示し、6 章にて、その実験結果を提示する。7 章では、性能差の要因, 512 バイト制約の影響, 誤分類分析, 運用上の観点から考察し、8 章にて、結論と今後の課題を述べる。

2 関連研究

2.1 ファイル断片識別とファイルカービング

ファイルカービングは、ファイルシステムのメタデータに依存せず、コンテンツ (メモリブロック) に基づいてファイルを識別・抽出する手法である [2], [5], [6]。具体的には、媒体上のデータ (バイト列) から、削除ファイルを未割り当て領域から復元したり、コンテナファイルから埋め込みファイルを抽出したりする技法である。最も単純な方法はマジックナンバー等の特徴的なデー

¹DFRWS による公開コンペティション DFRWS 2006 Forensics Challenge など。

²本論文では扱わないが、PDF のうち PDF 1.4 や PDF 1.7, JPEG のうち ベースライン形式やプログレッシブ形式といった粒度のもののファイル形式 (File format) と呼ぶ。なお関連して、先行研究の FiFTy は File Fragment Type の略であることを申し添えておく。

³混同行列の定義は 5 章 4 節 1 項に後述している。

タ列をキーとして検索し、キーが現れた箇所から連続するバイト列をファイルやデータレコードとして保存する [7]。なお、ここでの媒体とは、デジタルフォレンジクスの文脈であるから、データ（バイト列）がセクタやページなどの単位で記録され、物理的または論理的に記録・保持される解析対象となり得る記録対象を指す。例えば、磁気記憶装置や、半導体メモリ、光学記憶装置、仮想記憶（仮想メモリ）、あるいはやや拡張的にコンテナファイルなどが挙げられる。

ただしこれは、断片化が進んだ状況や、対象断片がヘッダーを含まない状況では適用が難しい。このため、ヘッダーに依存しないランダムな位置から切り出された小さなファイル断片（File fragment）から、断片の由来ファイル種別を識別する分類器が要求される。

2.2 手工特徴量に基づくファイル断片分類

従来の分類器は、連続バイトの変化率（RoC）やバイナリ頻度分布（BFD）[8]、コルモゴロフ複雑度[9]、バイト頻度分布（unigram / bigram）[10]、エントロピーなどの統計量[11]を設計し、SVM等の分類器に入力する枠組みが主流であった。これらは少数クラスや低エントロピーな形式では一定の効果を示す一方、高エントロピーな形式（圧縮・暗号化・実行形式等）では識別が難しい⁴。

具体的な代表例として Beebe らの Scedan [11] は、多様な統計特徴量を用いた SVM ベースの分類器であり、後続研究における強いベースラインとして扱われている。FiFTy 論文 [2] では、同一条件での再評価として、Scedan [11] が断片長 4096B の 75 クラスで 69.0% の精度を示す一方、特徴種類を増やすと精度が下がったことに加え、処理速度は FiFTy より 14 倍遅かったと報告されている [2]。

2.3 深層学習によるファイル断片分類

近年は深層学習による自動特徴抽出が検討されている。ただし、初期の研究では扱うファイル種別数が少ない、あるいはバイト列を one-hot 化して入力次元が過大になる等の制約があり、現実的な規模・多様性を備えた条件での検証が十分でない点が課題として残っていた。

そこで Mittal らが提案した FiFTy [2] は、メモリフォレンジクスおよびファイルカービングを想定したファイル断片の種別識別に関する研究とその成果であり、入力として加工していないバイト列（raw byte block）を直接扱う 1 次元 CNN を中核に据える。特徴量を事前設計するのではなく、0-255 のバイト値を「語彙」とみなし、学習可能な埋め込み（Embedding）を導入した上で畳み込みにより判別に有効な局所遷移パターンを獲得する設計が採られている。

FiFTy [2] は「速度・精度・汎用性」を設計目的に掲げ、CNN 推論の高速性と学習可能表現によって、従来の手工特徴量ベース手法で問題となっていた特徴抽出コストの削減を狙う。

2.4 FFT-75 データセットの概要

この FiFTy [2] では、評価用データセットとして 75 種類のファイル種別からなる FFT-75 [3] が構築・公開されている。このデータセットは、ビットマップ画像（6 種）、RAW 画像（11 種）、動画（7 種）、音声（7 種）などを含む多様な 75 ファイル種別を対象とし、ファイルカービング用途を強く意識した構成である。

さらに FiFTy [2] は、運用シナリオの差を反映するため、クラス粒度の異なる 6 シナリオ（scenario #1~#6）を定義する。scenario #1 は 75 ファイル種別をすべて独立クラスとする最も一般的な設定であり、派生的により粗い用途別分類へ集約可能な「基準シナリオ」と位置づけられる。一方、#3~#6 はメディア関連（写真・動画・JPEG など）を意識して、メディア関連の種別を細分化し、その他を集約する構成を持つ。

断片の大きさについても、ファイルシステムのクラスタサイズ等の差異を踏まえ、512B および 4096B の 2 条件が用意されている。FiFTy [2] の報告では、512B, scenario #1（75 クラス）において accuracy 65.6%、4096B 入力で accuracy 77.5% が示され⁵、この値は以後の比較研究における参照点となる。特に本研究では、512B, scenario #1 を扱う。

これは、データストレージ製品を開発・製造する企業 Western Digital による Advanced Format に関する白書 [12] にあるように、512B は Advanced Format が導入される以前からハードディスクのデータ格納セクタ長として歴史があることや、その互換性を踏まえて、Advanced Format による物理セクタが 4K に拡大された現在でもドライブ・インタフェース上は 512B をエミュレートしていることが由来である。つまり、実際に 512B 単位のメモリアクセスが起き得るという意味で、512B は現実的なサイズである。

FiFTy 論文 [2] では、FFT-75 [3] の性質として、DOC/PDF/PPT 等に画像が埋め込まれている場合があり、これが JPEG/PNG との混同を誘発し得る点が述べられている。この種の依存関係は、短い断片からの推定という問題設定において誤分類分析を行う際の重要な観点となる。さらに、実験の背景を踏まえファイルヘッダーが含まれ得ることに注意が必要である。

以上を踏まえると、FiFTy [2] は FFT-75 を用いた大規模・多クラスのファイル断片分類を整備し、CNN と Embedding を組み合わせた高速な基準手法を提示した点に意義がある。一方で、入力が系列データ（バイト列）であることか

⁴本研究の考察でも述べているが、高エントロピーな形式は特徴が分かりづらく、識別が難しいと考えられる。

⁵本論文に対応させると accuracy は test Top-1 accuracy である。

ら、系列モデリングに強みを持つ LSTM/GRU/Transformer といった自然言語処理系モデルの有効性を、同一データセット・同一シナリオ（特に 512B, scenario #1）で体系的に比較する余地が残る。本研究はこの比較を通じて、短いバイト列断片に対して「局所畳み込み中心の設計」と「系列依存関係中心の設計」のどちらが有効かを整理することを狙う。

3 問題設定とデータセット

3.1 問題設定（タスク定義）

本研究が対象とするのは、固定長 512 バイトのファイル断片（File fragment）から、その断片が由来するファイル種別（File type）を識別する多クラス分類問題である。入力を 512 バイトからなる列として、次のように表す。

- 入力: $x = (b_1, b_2, \dots, b_{512})$
 - 各 b_i は 1 バイトの値であるから、0～255 の整数となる
- 出力: $y \in \{0, 1, \dots, 74\}$
 - y は 75 種のファイル種別に対応するクラス ID である

したがって本タスクは closed-set（既知 75 クラスのみ）における多クラス分類であり、学習により得られる分類器 f は、入力断片 x に対して各クラスのスコア (logits) を出力し、最尤クラスを予測ラベルとして返す。評価は Top-1 accuracy を中心に行い、補助的に Top-3 accuracy や Macro-F1 等も用いる（指標の定義は 5 章に後述した）。

3.2 データセット FFT-75 と scenario #1

本研究では、先行研究 FiFTy [2] が公開した FFT-75 データセット [3] を用いる。FFT-75 [3] は 75 種類のファイル種別を対象とし、各サンプルは「単一ファイルから切り出した固定長（512B または 4096B）断片」と「その由来ファイル種別ラベル」から構成される。

FFT-75 [3] では運用上の要請に応じた複数のラベル設計（scenario #1～#6）が定義されるが、本研究は最も基本的な scenario #1 の 512 バイト長を採用する。512B, scenario #1 は 75 種別をすべて独立クラスとして扱う設定であり、以後の用途別集約（粗い分類）に対する基準点として解釈しやすい。ラベル体系（クラス ID と人間可読ラベルの対応）は、データセット付属の classes.json を基準として確定する。

3.3 断片生成仕様（512B, ランダム位置）

FFT-75 [3] の 512B 条件では、各サンプルは 512B 連続断片として与えられる。断片はファイルヘッダーに限定されず、ファイル内部の任意位置から切り出されることを前提としているため、マジックナンバーを含む場合と含まない場合が混在し得る。これは実運用（断片化・未割当領域・メモリブロック等）で観測される断片の状況

を反映するためであり、ヘッダーに依存しない識別が常に可能とは限らない条件を含んだベンチマークとなる。

また FiFTy [2] では、PDF/DOC/PPT 等のファイルの中に異なる形式のファイル（主に画像データ）が埋め込まれているコンテナファイルの場合があり、これが JPEG/PNG との混同を誘発し得ることが指摘されている [2]。したがって本タスクは、単純なファイルシグネチャ照合で一意に解ける状況だけを想定せず、「内部構造や埋め込みデータに起因する局所パターンの類似」も含めて分類器の頑健性が問われる。

3.4 データ分割（train/val/test）

本研究では FFT-75 [3] 付属の分割に従い、訓練データ（train）・検証データ（validation; val）・テストデータ（test）の 3 分割を用いる。本研究の実装では、FFT-75 のファイル構成を保持し、メモリマップ(memmap)として遅延ロードすることで、学習時のメモリ使用量を抑制した。各 split のサンプル数は以下である（512B, scenario #1）。

- train: 6,144,000
- val: 768,000
- test: 768,000

クラス数は 75 であり、テストデータにおける 1 クラス当たりのサンプル数は平均 10,240 で、最小 10,030、最大 10,391 と概ね均等に配分されている。本研究の最終比較では訓練データを全量（6,144,000）用いて学習し、検証データは早期終了やモデル選択の判断に用い、テストデータは最終性能の報告にのみ用いる。

3.5 入力表現と前処理（0-255, Embedding, padding/mask）

本研究では、入力断片の各バイト値（0～255）を「語彙サイズ 256 の離散トークン」とみなし、埋め込み層により連続ベクトルへ写像してから各モデル（CNN/LSTM/GRU/Transformer）へ入力する。このとき、バイト値を実数として正規化（例: 0～1 へ線形変換）するのではなく、離散シンボルとして扱うことで、バイト値の大小関係に依存しない表現学習を可能にする。

512B 条件では系列長が固定であるため、本研究の学習・評価では padding および attention mask は用いない。本研究は固定長条件に限定することで、モデル間の比較における実装差（padding/mask の流儀）を排し、系列モデルそのものの有効性に焦点を当てる。

4 提案手法

系列モデル（LSTM/GRU/Transformer）と CNN ベースラインのモデル構成を定義する。入力は 512 バイト断片を 0-255 の整数列として扱い、出力は 75 クラスそれぞれに対応ラベルである。モデル間の比較可能性を高めるため、入力表現（Embedding）、損失（Cross Entropy）、お

よび正則化 (Dropout 等) を共通化し、主たる差分が「局所パターン抽出 (CNN)」と「系列依存関係のモデル化 (RNN / Self-Attention)」になるよう設計する。

4.1 記法と入出力

バッチサイズを B , 系列長を T とする。本研究では入力断片長は固定であり $T = 512$ である。

入力断片は 1 バイト値 (0-255) からなる系列として表し, $x \in \{0, 1, \dots, 255\}^T$ とする。また, 1 サンプルのファイル種別ラベル (クラスラベル) を y として表し, そのラベル全てである 75 クラスの集合を $\mathcal{Y} = \{0, 1, \dots, 74\}$ として表す⁶。

このとき, 本研究で学習する分類器は, 入力系列 x をクラスラベル y に写像する関数

$$f_\theta : \{0, 1, \dots, 255\}^T \rightarrow \mathcal{Y} \quad (1)$$

を求める問題として定式化できる。なお, ここでは θ はモデルパラメータを示す。

実装上は, f_θ はクラスごとのスコア (ロジット) を出力する写像

$$f_\theta : \{0, 1, \dots, 255\}^T \rightarrow \mathbb{R}^{75} \quad (2)$$

として定義し, これによる出力を

$$z = f_\theta(x) = (z_0, \dots, z_{74}) \quad (3)$$

と書く⁷。

予測ラベル $\hat{y}$ は

$$\hat{y} = \arg \max_{k \in \mathcal{Y}} z_k \quad (4)$$

により得る⁸。学習時は softmax を明示的に計算せず, 損失は交差エントロピー (cross_entropy) を用いてロジットから直接計算する (損失の詳細は 5 章)。

4.2 共通設計 (Embedding・正則化・初期化・分類ヘッド)

本節の目的は, モデル族に依存しない設計要素を可能な限り共通化し, 性能差の主因が「系列モデリング機構そのもの」になるように比較条件を整えることである。

まず, バイト値の埋め込み (Embedding) について述べる。バイト値 0-255 を語彙サイズ 256 の離散トークンとみなし, 埋め込み層で連続ベクトルへ写像する。埋め込み次元は $E = 64$ とし, 入力

(B, T) を (B, T, E) へ変換する。実数正規化 (例: 0-1 への線形変換) ではなく離散的埋め込みを採用する理由は, 次の 3 点である。

1. バイト値の大小関係が必ずしも意味を持たないこと
2. 形式固有のパターンを学習により獲得させたいこと
3. 以後の系列モデル (RNN / Transformer) と整合しやすいこと

加えて, Embedding を共通化することで, 入力表現の差が比較結果へ与える影響を抑えることを意図している。また実装上は, Embedding 直後に時系列方向でマスクを共有する LockedDropout を任意に適用できる設計としている。本研究の既定設定では embedding_dropout=0.0 とし無効化しているが, 比較条件を保ったまま入力表現直後の正則化を導入できる拡張点として位置づける。

次に, 初期化について説明する。これは先行研究 FiFTy [2] の実装方針に合わせるため, 初期化を Keras の規定に近づけた。具体的には, Embedding は一様分布 $U(-0.05, 0.05)$, Conv1d および Linear は Xavier uniform (glorot_uniform) を用い, bias は 0 で初期化した。これにより, PyTorch の既定初期化との差が比較結果へ与える影響を抑制した。すなわち, 初期化に起因する収束挙動の差をなるべく小さくし, アーキテクチャ差の解釈を容易にすることを狙っている。

最後に, 分類ヘッドと活性化について述べる。CNN と Transformer では, 中間層を持つ MLP ヘッド (Linear, LeakyReLU, Dropout, Linear の順) により最終分類を行う。活性化は LeakyReLU ($\alpha=0.3$) を用いることで, FiFTy [2] の設計に合わせる。一方, LSTM/GRU は最終層の隠れ状態を特徴ベクトルとして取り出し, Dropout の後に単一の Linear で 75 クラスへ写像する。これは, 系列表現の取得方法を揃え, 差分を明確化するためである。また, 分類ヘッドを過度に複雑化しないことで, 特徴抽出器 (CNN / RNN / Self-Attention) の差が表れやすい設計としている。

4.3 CNN ベースライン (FiFTy 派生 1D-CNN)

CNN ベースラインは, FiFTy [2] の基本思想である「局所的なバイト遷移パターンの抽出」に基づく 1D-CNN とする。ここでの狙いは, 断片中の短い規則性 (局所パターン) を畳み込みにより検出し, プーリングと集約により固定長の判別特徴へ落とし込むことである。また, 後述の系列モデルと比較したときに, 「局所パターン中心の設計」がどこまで有効かを確認する基準として位置づける。forward の流れは, 埋め込みののちに畳み込みとプーリングを反復し, GAP により系列方向を集約した表現を MLP ヘッドで分類する構成である。全体構成を以下に示す。

1. Embedding : 入力 (B, T) を (B, T, E) に変換する

⁶つまり $y \in \mathcal{Y}$ である。

⁷ $\mathbb{R}$ は実数全体を示しているのので, 厳密に書くと $\mathbb{R}^{75} = \{(z_0, \dots, z_{74}) \mid \forall k \in \{0, \dots, 74\}, z_k \in \mathbb{R}\}$ である。

⁸arg max は, 一番大きい要素の添字 (クラス ID) を返す演算であり, 下つきの $k \in \mathcal{Y}$ は k の探索範囲を $\mathcal{Y}$ に限定する制約を示す。例えば, $z_{10} = 1.2, z_{23} = 3.7, z_5 = 2.1, \dots$ のように出たとき, 最大の z_{23} を持つクラス 23 を予測として採用し, $\hat{y} = 23$ となる。

2. 転置: (B,T,E) を (B,E,T) に並べ替える (Conv1d 入力形状へ変換)
3. 畳み込みブロック (num_blocks 回): Conv1d(in=E or C, out=C, kernel=K, stride=1, padding=valid) を適用したのち, LeakyReLU(0.3) と MaxPool1d(P) を順に適用する
4. Global Average Pooling (AdaptiveAvgPool1d(1)) により時系列長を 1 へ集約
5. MLP head: Linear(C, H), LeakyReLU, Dropout, Linear(H, 75) の順に適用する

本研究の既定設定は $E=64$, $C=128$, $K=27$, $P=4$, $\text{num_blocks}=1$, $H=256$, $\text{dropout}=0.1$ である. padding は「valid」(padding=0) とし, FiFTy [2] の Keras 実装における既定挙動に合わせる. また, Embedding 後に (B,T,E) から (B,E,T) へ転置するのは, Conv1d がチャンネル次元を第 2 軸に取る実装であるためである. さらに実装では, $\text{num_blocks} > 1$ の場合に限り, 最終ブロック以外の畳み込みブロック後へ dropout を挿入できる構成としている. また, GAP 直前に LayerNorm を挿入する実装上の選択肢があるが, 本研究の既定設定では無効化している.

系列長の変化を具体化するため, $T=512$ のとき, 畳み込み後の長さ T' は $T' = T - K + 1 = 486$ となる. これに MaxPool1d($P=4$) を適用すると, 時系列長は概ね $\frac{486}{4}$ に縮小され (端数は切り捨て), その後 GAP により 1 へ集約される. したがって CNN は「局所受容野 K の特徴抽出」と「プーリングによる短縮」を経て, 固定長特徴ベクトル (チャンネル数 C) を得る設計である.

4.4 LSTM ベースモデル

LSTM ベースモデルは, 系列の長短依存を再帰状態により表現する. 構成は次の通りである. 本研究では, バイト列を「順序を持つ信号」とみなし, 畳み込みよりも系列依存の蓄積に重きを置く設計として LSTM を採用する. 特に双方向化により, 断片内の前後双方の文脈を同時に参照できる点を重視した. forward の流れは, 埋め込み表現を Bi-LSTM により系列変換し, 最終状態を連結して得た表現を線形層で分類する構成である.

1. Embedding: 入力 (B,T) を (B,T,E) に変換する
2. LSTM (多層 Bi-LSTM): $\text{input_size}=E$, $\text{hidden_size}=H_{\text{rnn}}$, $\text{num_layers}=L$
3. 最終層の最終状態 h_n を用いた系列集約
4. Dropout ののち Linear を適用してロジットを得る

本研究では双方向 (bidirectional=true) を採用する. 双方向の場合, 最終層の forward/backward の最終状態を連結し, 特徴ベクトルを $v \in R^{2H_{\text{rnn}}}$ とする. 単方向の場合は $v \in R^{H_{\text{rnn}}}$ であるが, 本研究は比較条件を統一するため双方向固定とする. 実装上は, 双方向時には最終層の 2 方

向ぶんの最終状態のみを取り出して連結し, 全時刻の出力列は分類器へ渡さない設計としている.

系列集約として「全時刻の平均」や「attention pooling」等も考えられるが, 本研究ではモデル間差分を明確化するため, RNN の標準的取り出しである最終状態 (h_n) を採用する. これは, 系列の要約方法そのものが性能差の要因になることを避け, 再帰ユニットの違いに主眼を置いて比較するためである.

4.5 GRU ベースモデル

GRU ベースモデルは, LSTM と同様の枠組みで再帰ユニットのみを GRU に置換する. GRU は LSTM に比ベゲート構造が簡略であり, パラメータ数および計算量が小さくなる傾向がある. このため, 同程度のパラメータ規模にそろえた条件では, 最適化の安定性や表現効率の差が結果に現れることが期待される. 本研究では, LSTM と同一の「Embedding, RNN (多層 Bi-RNN), 最終状態による集約, 分類」という枠組みを保つことで, 再帰ユニットの違いが性能へ与える影響を検証する. なお実装上は, GRU/LSTM いずれも $\text{num_layers} > 1$ のときに限り, 層間 dropout が RNN 内部で有効になる設定としている. forward の流れは, 埋め込み表現を Bi-GRU により系列変換し, 最終状態を連結して得た表現を線形層で分類する構成である.

4.6 Transformer ベースモデル

Transformer ベースモデルは, 自己注意 (Self-Attention) により系列内の広域依存を直接モデル化する. 本研究の Transformer は「Encoder のみ」を用い, 因果マスク (未来遮断) は適用しない. 入力が固定長かつ分類問題であるため, 全時刻相互参照を許容する設定とする. また, CLS トークンを導入することで, 系列全体の情報を 1 つの位置表現へ集約し, 分類ヘッドへの入力を明確化する設計とした. forward の流れは, 埋め込みののちに CLS トークンを付与して位置情報を加え, Transformer エンコーダを通したうえで CLS 位置の表現を抽出し, MLP ヘッドで分類する構成である.

構成は次の通りである.

1. Embedding: 入力 (B,T) を (B,T,E) に変換する
2. CLS トークン付与: 学習可能パラメータ $c \in R^{\{E\}}$ をヘッダーに連結し, 系列長を $T+1$ とする
3. 位置エンコーディング: 正弦波により位置情報を加算する
4. Transformer エンコーダー (L 層):
 - $d_{\text{model}} = E$ (埋め込み次元)
 - $n_{\text{head}} = H_{\text{attn}}$ (注意ヘッド数)
 - $d_{\text{ff}} = F$ (FFN 中間次元; dim_feedforward)
 - 活性化: GELU
 - Pre-LN ($\text{norm_first}=\text{true}$)

- 出力のうちCLS位置(ヘッダートークン)の表現を特徴ベクトルとして取り出す
- MLP head: Linear(E, H), LeakyReLU, Dropout, Linear(H, 75)の順に適用する

本研究の既定設定は $E = 64, L = 5, H_{\text{attn}} = 4, F = 256$, head の中間次元 $H = 256$, dropout = 0.1 である。位置エンコーディングに正弦波を採用する理由は、学習型位置埋め込みに比べ追加パラメータが増えず、初期条件の差を抑えやすい点にある。さらに Pre-LN (norm_first=true) は、深い層数での学習安定性に配慮した選択である。実装上は、CLS トークンを先頭に連結したのちに位置エンコーディングを加算し、エンコーダ出力の先頭位置 (CLS 位置) のみを特徴として分類ヘッドへ入力している。また、Transformer エンコーダでは nested tensor を無効化しており、固定長入力における挙動の単純化を優先している。

4.7 パラメータ規模の整合 (比較の公平性)

モデル間で表現力の差が過度にならないよう、埋め込み次元 E と dropout を共通化し、RNN の hidden_size 等を調整してパラメータ数を概ね同規模に揃える。本研究の既定設定における総パラメータ数は以下の通りである。

- CNN: 289,995
- GRU: 291,211
- LSTM: 288,535
- Transformer: 302,283

Transformer が他よりやや多いが、いずれも約 30 万規模であり、「モデル族の差」ではなく「系列モデリング機構の差」を主として比較できる条件を目標とする。また、初期化手法を統一し、入力表現を共通化することで、学習の初期挙動や収束の差がモデル固有の構造差に由来するよう配慮する。

5 実験設定

5.1 実験環境

実験は GPU を用いて実行する設定とし、学習・評価は同一の実装により各モデル種別を個別に学習した。設定ファイルでは、実行対象モデル種別として cnn/gru/lstm/transformer を列挙し、乱数シードは 42 に固定した。

データセットは FFT-75 [3] の .npz 形式 (x.npz, y.npz) をそのまま使い、学習時にはメモリマップ (mmap) として読み込むことで、データ全量 (train/val/test) を扱う場合でもホストメモリ消費を抑制する設計とした。

GPU 型番, CPU, RAM, OS, Python/PyTorch/CUDA などの、実行環境および主要ソフトウェアは付録に示した。

5.2 実験手順

1つのモデル種別 (run_type) について、次の順に実行した。

1. データ split の読み込み

2. 学習ハイパーパラメータの確定
3. モデル初期化
4. 学習 (検証を挟みつつチェックポイント保存)
5. テスト評価
6. 混同行列⁹とレポート生成
7. 学習曲線の可視化出力

検証 (val) は val_interval=1 とし、原則として各エポック終了時に実施した。探索フェーズ (phase=search) では検証を割合サブセットで行う実装であるが、本研究の既定設定は phase=final である。

検証は全量で実行した。すなわち、実装上は val_max_batches 指定がある場合はその上限が利用されるが、本研究では指定しなかったため、検証データ全量を用いた。

5.3 学習条件

共通設定としてハイパーパラメータのうち、バッチサイズは 128, エポック数は 48, 正則化は weight_decay=1.0e-6, 勾配ノルムクリッピングは clip_grad_norm=0.5 とした。ラベル平滑化 (label_smoothing) は LSTM/GRU/Transformer で 0.01, CNN では 0.0 を採用した。

訓練データは n_subset=6,144,000 (100%) を用い、各エポックで訓練サンプルの順序を乱択した。乱択は (seed + epoch) で初期化した乱数生成器により randperm を生成して行い、同一 seed 条件下でエポックごとのシャッフルが再現されるようにした。

最適化手法は adamw を用い、学習率は探索の結果として次を採用した。学習率スケジューラは scheduler=cosine とし、warmup_epochs=3, warmup_start_factor=0.1, eta_min=1.0e-7 を用いた。

- CNN: 0.0010
- GRU: 0.0006
- LSTM: 0.0001
- Transformer: 0.0010

混合精度学習 (AMP) は有効化した。具体的には autocast と GradScaler を用いてスケールン付きで逆伝播を行った。

損失関数は多クラス交差エントロピー誤差 (Cross-entropy Loss) を用いた。具体的には、実装上は label_smoothing を引数として受け取り、LSTM/GRU/Transformer では 0.01, CNN では 0.0 を用いた。したがって CNN は通常の交差エントロピーと等価であり、他モデルは軽微な平滑化を含む。訓練時には Top-1 accuracy に加え、Top-3 accuracy を併記した (指標の定義は 5 章に後述した)。

5.4 検証・評価

検証の範囲は、次の 2 系統で計算量を制御できるように設計した。

⁹混同行列の定義は 5 章 4 節 1 項に後述している。

1. val_max_batches による評価バッチ数上限
2. val_subset_ratio によるサンプル割合サブセット

例えば, val_subset_ratio を指定した場合は, val_subset_seed によりサブセット抽出が再現可能となる設計にした。

なお, 早期終了(Early Stopping)は, 評価指標を val_loss を最小化するように設定した。これは, 評価指標を最小化 (mode=min) するときには次の式で,

$$\text{metric_value} < \text{best} - \text{min_delta} \quad (5)$$

評価指標を最大化 (mode=max) のときするときには次の式

$$\text{metric_value} > \text{best} + \text{min_delta} \quad (6)$$

を満たす階数が patience=6 続いた場合に学習を打ち切った。なお, metric_value は現在の評価指標の値 (今回は val_loss), best はその評価指標で過去に出た最も良い値 (今回は最も小さい値), min_delta は改善とみなす最小差分 (今回は min_delta=0.0) を意味する。

また, 各エポック終了時点でチェックポイント (latest) を保存し, 早期終了指標が改善した場合には best チェックポイントを更新した。このとき, 学習中に損失が非有限 (NaN/Inf 等) となると, 最新チェックポイントへ巻き戻し, 学習率を 1/2 に減衰させた上で当該エポックを再実行 (再試行回数には上限を設ける) した。

評価 (val/test) では, 交差エントロピー誤差, Top-1 accuracy, Top-3 accuracy および Macro-F1 を算出した。

Top-1 accuracy は, モデルが出力した分類スコアのうち最大値を持つクラス¹⁰を予測結果とみなし, その予測が正解ラベルと一致した場合を正解とみなし, その割合を表す指標とした。一方 Top-3 accuracy は, モデルが出力した分類スコアの最上位 3 クラスそれぞれを予測結果とみなし, その中に正解ラベルと一致するクラスが含まれている場合を正解とみなし, その割合を表す指標とした。

また, Macro-F1 は, テスト評価時の混同行列 (定義は 5 章 4 節 1 項に後述した) からクラスごとの F1 を算出し, その平均を取った。ここでクラス k の F1 は, 混同行列に基づく TP_k, FP_k, FN_k を用いて $F1_k = 2 \frac{TP_k}{2TP_k + FP_k + FN_k}$ とし, 分母が 0 の場合は 0 とした。

さらに, FiFTy 論文で指摘される, コンテナファイル (PDF/DOC/PPT 等) と画像 (JPEG/PNG) の混同を補助的に観測するため, 設定可能な場合は JPEG クラスに限定した正答率 jpeg_acc と, その評価対象サンプル数 (support) を併記した。

¹⁰Top-3 accuracy の説明に倣うと, 最上位 1 クラスである。

5.4.1 テスト評価時の混同行列

バッチサイズを B , 系列長を T とする。本研究では入力断片長は固定であり $T = 512$ である。

入力断片は 1 バイト値 (0–255) からなる系列として表し, $x \in \{0, 1, \dots, 255\}^T$ とする。また, 1 サンプルのファイル種別ラベル (クラスラベル) を y として表し, そのラベル全てである 75 クラスの集合を $\mathcal{Y} = \{0, 1, \dots, 74\}$ として表す¹¹。

テスト評価時の混同行列について説明する。定義上の混同行列はテスト集合 $\mathcal{D}_{\text{test}}$ 全体に対する集計として与えるので, $\mathcal{D}_{\text{test}}$ をまず定義する。

これは, 4 章 1 節で定義した次の記号

- バッチサイズ (実装上のミニバッチ処理に用いる): B
 - 系列長: $T = 512$
 - クラス集合 (全 75 クラス): $\mathcal{Y} = \{0, 1, \dots, 74\}$
 - 1 バイトの値の集合: $\{0, 1, \dots, 255\}$
- に加えて,

- n 番目サンプルの入力 (長さ T のバイト列): x_n
- それに対する正解ラベル (真のラベル; クラス ID): y_n
- テストサンプル総数: N_{test}

を用いると, テスト集合 $\mathcal{D}_{\text{test}}$ は N_{test} 個のサンプルからなる列として次で定義される。

$$\mathcal{D}_{\text{test}} = (x_n, y_n)_{n=1}^{N_{\text{test}}}, \quad (7)$$

$$x_n \in \{0, 1, \dots, 255\}^T, \quad y_n \in \mathcal{Y}$$

なお, 実装では $\mathcal{D}_{\text{test}}$ をバッチサイズ上限 B のミニバッチに分割して逐次処理し, 混同行列は全サンプルの集計として算出する。これは, ミニバッチ分割に関する次の記号

- 実際に処理したバッチ総数: M
- バッチ番号: $m \in \{1, \dots, M\}$
- バッチ m の実サンプル数: B_m
- バッチ内での添字: $b \in \{1, \dots, B_m\}$
- バッチ m の b 番目入力: $x_{m,b}$
- それに対する真のラベル: $y_{m,b}$

を導入すると, テスト集合 $\mathcal{D}_{\text{test}}$ はミニバッチに分割されるので次で表される。

$$\mathcal{D}_{\text{test}} = \bigcup_{m=1}^M \{(x_{m,b}, y_{m,b})\}_{b=1}^{B_m}, \quad (8)$$

$$x_{m,b} \in \{0, 1, \dots, 255\}^T, \quad y_{m,b} \in \mathcal{Y}$$

このとき, 最終バッチでは $B_m < B$ となり得る。

モデルをパラメータ θ をもつ写像 f_θ としたときの各クラスのスコア (ロジット) z_n は式 (3) より,

$$z_n = f_\theta(x_n) = (z_{n,0}, z_{n,1}, \dots, z_{n,74}) \in \mathbb{R}^{75} \quad (9)$$

¹¹つまり $y \in \mathcal{Y}$ である。

で与えられる。ここで、 $z_{n,k}$ は、サンプル n に対するクラス k のロジットである。また、予測ラベル $\hat{y}_n$ は式 (4) で定義される。

次に、混同行列を $C \in \mathbb{N}_0^{75 \times 75}$ とし、その (i, j) 要素 C_{ij} を真のラベルが i 、予測ラベルが j となった件数として次で定義する。

$$C_{ij} = \sum_{n=1}^{N_{\text{test}}} \mathbf{1}[y_n = i] \mathbf{1}[\hat{y}_n = j], \quad i, j \in \mathcal{Y} \quad (10)$$

ここで $\mathbf{1}[\cdot]$ は指示関数であり、条件が真のとき 1、偽のとき 0 である。また、行 i は真のクラス i を、列 j は予測クラス j を示す。

なお、本実装で保存される混同行列は正規化前の生カウントであり、さらに、

$$\sum_{i \in \mathcal{Y}} \sum_{j \in \mathcal{Y}} C_{ij} = N_{\text{test}} \quad (11)$$

が成り立つ（全サンプルがちょうど 1 回だけ、いずれかの (i, j) に加算される）ことを検証している。

さらに、クラス $k \in \mathcal{Y}$ に対する TP/FP/FN は、混同行列 C を用いて次で与える。

$$\begin{aligned} \text{TP}_k &= C_{kk}, \\ \text{FP}_k &= \sum_{i \in \mathcal{Y}, i \neq k} C_{ik}, \\ \text{FN}_k &= \sum_{j \in \mathcal{Y}, j \neq k} C_{kj} \end{aligned} \quad (12)$$

これらから、クラス k の Precision / Recall / F1 を

$$\begin{aligned} \text{Precision}_k &= \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \\ \text{Recall}_k &= \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \\ \text{F1}_k &= 2 \text{Precision}_k \frac{\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \end{aligned} \quad (13)$$

で定義する。ただし分母が 0 となる場合 ($\text{TP}_k + \text{FP}_k = 0$ または $\text{TP}_k + \text{FN}_k = 0$ など) は 0 とする

6 実験結果

各エポック終了時に検証データ (validation) を評価し、検証損失 (validation loss; val_loss) の最小化を基準として最良エポックを選択し、学習終了時にテストデータで最終評価を行った結果¹²を示す。評価指標は、5 章 4 節で述べた通り、損失 (cross-entropy loss)、Top-1 accuracy、Top-3 accuracy、Macro-F1 を用いる。

¹²early stopping (metric=val_loss, mode=min, patience=6, restore_best=true) は有効化した。

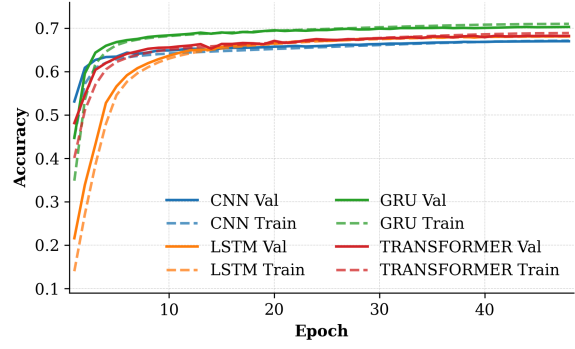


図 1: 各モデルの学習曲線 (Top-1 accuracy)

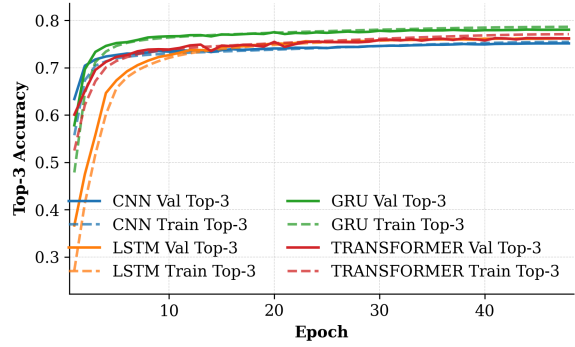


図 2: 各モデルの学習曲線 (Top-3 accuracy)

6.1 総合性能 (検証データ・テストデータ)

最良エポックにおける検証データに対する指標を表 1 に、テストデータにおける指標を表 2 に示す。

また、正答率の曲線を図 1 および図 2 に示す。

本実験 (48 epoch 設定) では、GRU が最も高い性能を示し、テストデータに対して Top-1 accuracy 70.1% を達成した。これは CNN ベースライン (66.9%) に対して +3.2pt, LSTM (67.9%) に対して +2.2pt, Transformer (68.1%) に対して +2.0pt の改善である。Top-3 accuracy および Macro-F1 も同様に GRU が最大となり、クラス間で均等に近い分布を持つデータセットにおいて

表 1: 検証データに対する各モデルの性能指標

Model	Top-1	Top-3	Loss	Macro-F1
CNN	0.6698	0.7515	1.1747	0.6663
GRU	0.7027	0.7805	1.0435	0.7035
LSTM	0.6810	0.7615	1.1245	0.6796
Transformer	0.6823	0.7627	1.1481	0.6811

表 2: テストデータに対する各モデルの性能指標

Model	Top-1	Top-3	Loss	Macro-F1
CNN	0.6690	0.7510	1.1773	0.666
GRU	0.7010	0.7790	1.0520	0.7030
LSTM	0.6790	0.7610	1.1310	0.6780
Transformer	0.6810	0.7600	1.1634	0.6780

て、Top-1 の改善がクラス平均指標にも反映されていることが確認できる。

なお、混同行列から得られるテストデータに対する各モデルのクラス別指標は、CNN, GRU, LSTM, Transformer それぞれについて、付録の表 appx.1, 表 appx.2, 表 appx.3, 表 appx.4 に示す。

また、混同行列を図示したグラフは、CNN, GRU, LSTM, Transformer それぞれについて、付録の図 appx.1, 図 appx.2, 図 appx.3, 図 appx.4 に示す。

このグラフは、縦軸が真のクラス、横軸が予測クラス、色の濃さ (Count) が件数 (サンプル数) を表す。対角成分が大きいほど正分類件数が多く、非対角成分はクラス間の誤分類傾向 (取り間違えた件数) を示す。「全て間違わない場合」の理論上の見え方は、非対角成分がすべて 0 で、対角成分 (i, i) が各クラスのサンプル数 N_i になる。このとき 1 セルの理論上の最大値は $\max_i N_i$ (評価データ中で最も件数の多いクラスのサンプル数) である。

7 考察

7.1 総合性能差と系列モデル適用の含意

本研究の 512B, scenario #1 (75 クラス) 条件では、test Top-1 accuracy は表 2 より GRU 70.1%, Transformer 68.1%, LSTM 67.9%, CNN 66.9% の順であった。test Top-3 accuracy は表 2 より GRU 77.9%, LSTM 76.1%, Transformer 76.0%, CNN 75.1% の順であった。どちらも先行研究の FiFTy [2] を再現した CNN より、系列モデルのほうが一貫して優位であり、LSTM と Transformer は同程度の精度であった。クラスが概ね均等であるため Macro-F1 は Top-1 と近い値となり、順位もほぼ同一であった。なお、テストデータのサンプル数は 768,000 と大きく、Top-1 の差分 1~6pt は標準誤差¹³が約 0.05pt 程度であることから、偶然誤差のみで説明するのは難しい。

CNN は局所受容野 (kernel=27) に基づく局所遷移パターン抽出に強みがあるが、512B 断片では「局所パターンだけでは決めきれない」ケースが一定量残る。これに対し、GRU/LSTM は系列としての状態遷移を通じてより広い文脈を統合でき、短断片でも「局所特徴の組合せ」を扱いやすいことが、CNN との差 (約 2~4pt) として現れた可能性がある。また、比較の公平性の観点では、埋め込み次元等を共通化し、各モデルの総パラメータ数を約 30 万規模に揃える設計が採用されている。したがって本実験で観測された差は、主として「系列モデリング機構 (ゲート付き再帰, 自己注意, 畳み込み) の違い」が 512B 断片の識別に与える影響として解釈できる。

具体的に、GRU が相対的に有利となった要因として、次の 3 つが挙げられる。

1. 512 ステップという短系列に対して、ゲート機構が局所パターン (バイト列の短い規則性) と弱い長距離依存 (同一断片内に散在する構造手掛かり) を過不足なく保持できた可能性
2. LSTM よりもパラメータ効率が良く、学習の安定性や正則化なし条件での最適化が有利に働いた可能性
3. Transformer は一般に学習率スケジュールやウォームアップ等の設計に敏感であり、本研究の「単純化した学習条件」では能力を十分に引き出せなかった可能性
4. 512B 断片の識別に必要な手掛かりが超長距離依存よりも、局所~中距離の統計・構造に偏ることで Transformer の利点が相対的に出にくかった可能性

特に、本研究では学習率は CNN/GRU/LSTM/Transformer すべてで探索したもの采用了が、計算資源の都合 0.0001, 0.0003, 0.0006, 0.0010 の範囲でしか探索出来なかった。これはモデル間の条件を揃えることで単純化した比較ができる一方、各モデルにおける「構造としての限界」と「学習設定上の未最適化」を切り分けるには追加実験が必要である。

7.2 誤分類傾向と 512B 制約

上位の混同行列をみると、いずれのモデルでも 7z/xz/bz2 等の圧縮形式 (高エントロピー断片になりやすいクラス) に誤分類が集中する傾向が見られた。特に GRU/LSTM は bz2, CNN/Transformer は xz への誤分類が顕著であり、断片がヘッダー等の決定的手掛かりを欠く場合に「圧縮データに近い統計的性質」へ収束してしまう構図が示唆される。これは、短いバイト列断片を統計量として観測したときに、圧縮アルゴリズムが生成するデータと類似した特徴へ収束しやすいことを意味する。その一因として、多くの圧縮形式 (7z/xz/bz2 など) は、内部でエントロピー符号化 (Huffman, arithmetic coding 等) を用いるため、出力されるバイト列は 0~255 の値が比較的均等に出現する。これを踏まえると、断片長が 512 バイトと短い条件では、フォーマット固有のマーカーや構造的な手掛かりが断片に含まれない場合が多く、観測可能な特徴はバイト頻度分布やエントロピーといった低次の統計量に限定されるため、これらの統計量が前述の圧縮形式の出力特性と類似する断片については、予測が圧縮形式クラスへ偏りやすくなる。

また、クラス別指標をみても xz や exe の F1 が極端に低く (Transformer の xz が 0.011765) など¹⁴、全体精度だけでは見えにくい難クラスが存在が確認できる。これは、暗号化・圧縮等が施された高エントロピーな形式において、元のデータ

¹³独立試行の二項近似に基づく仮定の下である。

¹⁴各モデルの最も低い F1 は、CNN では xz が $F1=0.059206$, LSTM では exe が $F1=0.042320$, GRU では exe が $F1=0.056708$, Transformer では xz が $F1=0.011765$ であった。

を識別可能であることは、さらなる暗号化や圧縮が可能（必要）であることを意味するため、形式固有の規則性（シグネチャや構造的偏り）が断片中に現れにくく、短断片からの識別可能性が低下しやすいからである。したがって、この種の形式に対しては、入力長を固定した単一断片分類に基づく Top-1 精度の向上には限界が生じ得る。また File fragment が必ずしもファイルヘッダーを含まないことにも起因する。

jpeg_acc（ログに記録された JPEG 系ラベルの精度）は GRU 91.4%, LSTM 89.6%, CNN 88.6%, Transformer 90.6% と比較的高く、画像系の一部は短断片でも識別しやすいことが確認できる。FiFTy [2] が指摘する「コンテナファイル内の埋め込み画像による混同」は重要な観点であるが、本実験の上位混同はまず高エントロピー系の相互誤分類として現れており、512B 条件では「圧縮・暗号・実行・一部コンテナ」の判別がボトルネックになりやすい可能性が高い。

また、そもそも FFT-75 [3] の 512B 条件は、ファイルヘッダーに限定せず内部の任意位置から断片を切り出す前提であり、マジックナンバー（ヘッダー情報）を含む場合と含まない場合が混在し得る。この設定は、実運用に近い一方で、「ヘッダー依存の識別」が成立しないサンプルが必ず含まれるため、分類問題としての難易度上限を押し上げる。

7.3 バイト列表現設計の妥当性と今後

本研究は、各バイト値（0～255）を語彙 256 の離散トークンとみなし、埋め込み（Embedding）により連続ベクトルへ写像してから各モデルへ入力する。バイト値を 0～1 に正規化するのではなく離散シンボルとして扱う設計は、「大小関係が意味を持たないバイト値」に対して不要な順序構造を導入しない点で妥当である。また固定長 512B に限定することで padding/mask を用いず、モデル間の実装差を抑える方針も、比較実験として合理的である。一方で、この設計は「可変長断片」や「複数断片の統合」といった実運用寄りの条件へ拡張する際に、PAD トークン設計や mask の扱いが新たな性能差要因になり得る。今後の拡張では、入力長の揺らぎが系列モデル（特に Transformer）に与える影響も含めて再評価が必要である。

7.4 実運用上の示唆と研究上の限界

ファイルカービングの前処理としてファイル種別識別を用いる場合、誤分類は復元処理の探索空間を誤って広げ、誤検出（false positives）を増やす要因となる。難クラス（例: exe, 圧縮形式群）に対しては、次の 4 つのような対策が現実的である。

1. 単発の Top-1 を確定判断に使わず Top-k を併用する
2. 予測確信度が低い断片を unknown として保留する運用（open-set 的扱い）

3. 既知の強いシグネチャがある場合はシグネチャ照合と併用する
4. 同一領域から複数断片を取得できる場合は予測を統合する

また本研究は単一断片である 512B, scenario #1 に限定しており、断片長拡大（4096B）や可変長、シナリオ変更（用途別集約）により結論が変わる可能性がある。さらに、Transformer の最適化条件、RNN の勾配安定化、CNN のブロック数増加など、各モデルの到達可能な上限を比較する設計は今後の検討課題である。

また、研究上の限界としては次の 3 つが挙げられる。これらは今後の課題である。

1. 学習率やスケジューリング等がモデルごとに最適化されていない可能性
2. 単一条件（512B, scenario #1）に限定しており、4096B や他 scenario で順位が変わり得る点
3. 正則化なし・固定エポック上限など学習条件の単純化が各モデルに与える影響の切り分け

8 結論

本研究では、FFT-75 [3] の 512B, scenario #1（75 クラス）条件において、ファイル断片のファイル種別識別を対象とし、系列モデル（LSTM/GRU/Transformer）と CNN ベースラインを同一枠組みで比較した。入力は 0-255 のバイト値を語彙 256 の離散トークンとして埋め込み表現に変換し、固定長 512B の単一断片进行分类する設定とした。

実験の結果、test Top-1 accuracy は GRU 70.1%, Transformer 68.1%, LSTM 67.9%, CNN 66.9% であり、Top-3 accuracy は GRU 77.9%, LSTM 76.1%, Transformer 76.0%, CNN 75.1% であった。いずれの指標でも系列モデルが CNN を一貫して上回り、特に GRU が最も高い性能を示した。また、テストデータのサンプル数が 768,000 と大きい条件では、Top-1 の 1-6pt 程度の差は偶然誤差のみで説明しにくく、系列モデル適用の効果が示唆される。これらから、512B 断片のように入力が短い条件でも、局所パターンに加えて断片内の手掛かりの組合せを統合できる系列モデルが有効となり得る、という知見が得られた。

誤分類分析では、7z/xz/bz2 等の圧縮形式に誤分類が集中する傾向が確認された。断片がヘッダー等の決定的手掛かりを欠く場合、観測可能な特徴が低次の統計量（頻度分布・エントロピー等）に限定されやすく、高エントロピー断片同士の識別が難しくなることが要因として考えられる。実際、クラス別指標では xz や exe の F1 が極端に低いモデルが存在し、全体精度だけでは見えにくい難クラスが支配的なボトルネックになり得ることが分かった。一方で、JPEG 系の一部は高い精度で識別でき、短断片でも識別しやすい形式が存在することも確認された。

以上を総合すると、次の3つの結論に至る。

1. FFT-75 の 512B, scenario #1 において系列モデルは CNN ベースラインより高性能であった
 2. GRU は本条件下で最良の精度を示した
 3. 512B 単一断片分類では高エントロピー系(圧縮・暗号・実行形式等)が難所となった
- ただし、本研究は単一断片である 512B, scenario #1 に限定しているため、断片長の拡大(4096B)、可変長入力、他シナリオへの適用で結論が変化する可能性がある。また、モデルごとの最適化(正則化, Transformer の warmup 等)が十分ではなく、「構造としての限界」と「学習設定上の未最適化」を完全には切り分けられていない。したがって、本研究の結論は「本研究の統一条件における比較結果」として解釈する必要がある。

最後に今後の課題としては、次の5つが挙げられる。

1. 4096B や可変長断片での再評価
2. Transformer に対する最適化設計(AdamW・warmup・cosine decay・正則化)と到達性能の検証
3. 難クラス(圧縮・暗号・実行形式)の open-set (unknown) 判定や確信度校正
4. 複数断片の統合(投票・確率平均・系列化)と運用上の誤検出抑制
5. 混同行列上位ペアの系統的分析(コンテナ ↔ 画像, 圧縮形式相互など)による限界の定量化

謝辞

本研究にあたって、終始の御指導・御助言を賜りました、有明工業高等専門学校 創造工学科 人間・福祉工学系 Gauthier Lovic 教授¹⁵にこの場を借りて深く感謝いたします。

参考文献

- [1] R. R. Ali と K. M. Mohamad, 「RX_myKarve carving framework for reassembling complex fragmentations of JPEG images」, *Journal of King Saud University - Computer and Information Sciences*, vol. 33, no. 1, 2021, doi: <https://doi.org/10.1016/j.jksuci.2018.12.007>.
- [2] G. Mittal, P. Korus, と N. Memon, 「FiFTy: Large-scale File Fragment Type Identification using Neural Networks」. [Online]. 入手先: <https://arxiv.org/abs/1908.06148>
- [3] G. Mittal, P. Korus, と N. Memon, 「File Fragment Type (FFT) - 75 Dataset」. [Online]. 入手先: <https://dx.doi.org/10.21227/kfxw-8084>
- [4] D. Jurafsky と J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd 版. 2026.

¹⁵<https://researchmap.jp/gauthier>

[Online]. 入手先: <https://web.stanford.edu/~jurafsky/slp3/>

- [5] National Institute of Standards and Technology, 「CFReDS File Carving Dataset」. [Online]. 入手先: <https://cfreds-archive.nist.gov/FileCarving/index.html>
- [6] National Institute of Standards and Technology, 「Forensic File Carving Tool Specification」, Draft Specification, 2017. [Online]. 入手先: <https://www.nist.gov/system/files/documents/2017/05/09/fc-req-public-draft-01-of-ver-01.pdf>
- [7] 稔, 「フォレンジックツールの開発と実装」. [Online]. 入手先: https://www.ij.ad.jp/dev/tech/techday/2018/pdf/TD2018_2.pdf
- [8] M. Karresand と N. Shahmehri, 「File Type Identification of Data Fragments by Their Binary Structure」, *2006 IEEE Information Assurance Workshop*, 2006. doi: 10.1109/IAW.2006.1652088.
- [9] C. J. Veenman, 「Statistical Disk Cluster Classification for File Carving」, *Third International Symposium on Information Assurance and Security*, 2007. doi: 10.1109/IAS.2007.75.
- [10] S. Fitzgerald, G. Mathews, C. Morris, と O. Zhulyn, 「Using NLP techniques for file fragment classification」, *Digital Investigation*, vol. 9, pp. S44-S49, 2012, doi: <https://doi.org/10.1016/j.diin.2012.05.008>.
- [11] N. L. Beebe, L. A. Maddox, L. Liu, と M. Sun, 「Sceadan: Using Concatenated N-Gram Vectors for Improved File and Data Type Classification」, *Trans. Info. For. Sec.*, vol. 8, no. 9, pp. 1519-1530, 9 月 2013, doi: 10.1109/TIFS.2013.2274728.
- [12] Western Digital Corporation, 「Advanced Format Technology」, White Paper, 2011. [Online]. 入手先: https://documents.westerndigital.com/content/dam/doc-library/en_us/assets/public/western-digital/collateral/white-paper/white-paper-advanced-format.pdf

付録

A 実装済みの使用しなかった機能

4 章に関連して共通性能として実装したが本研究では無効化した機能として、Embedding 出力直後に時系列方向に対して、同一の脱落マスクを全時刻で共有する脱落処理(Locked Dropout)がある。通常の dropout は時刻ごとに異なるマスクが入り得るが、系列長が短い場合には情報の断絶を生みやすい。そこで、LockedDropout は特徴次元単位で一貫して無効化する性能を活かし、同じ特徴が時刻により出現の有無が変化する状況を緩和し、系列モデルの学習安定性の向上が期待される。ただし本研究では実験の複雑性が増すため `embedding_dropout=0.0` とし、実装はしたものの利用しなかった。そのため、影響評価は将来課題とする。

B 実行環境および主要ソフトウェア

本研究における実行環境および用いた主要なソフトウェアとその版数を以下に示す。これは、

実験結果に影響しうる計算資源を示すことで再現性と透明性を高めるためである.

B.1 実行環境

- CPU: Intel Core i7-9700K (8 cores, up to 4.90 GHz)
- GPU: NVIDIA GeForce RTX 2080 Mobile
- Memory: RAM 16 GB (15929MiB) DDR4-2666 (dual-channel)
- OS: Ubuntu 24.04.3 LTS (x86_64)
- Kernel: 6.8.0-90-generic
- Python 3.12.3
 - torch: 2.5.1+cu121
- CUDA Version: 12.2

B.2 使用ライブラリ

PyTorch 関連のライブラリとして次.

- torch: 2.5.1+cu121
- torchvision: 0.20.1+cu121
- torchaudio: 2.5.1

PyTorch のモデル可視化のライブラリとして次.

- torchinfo: 1.8.0
- torchview: 0.2.7

結果のグラフ生成や設定ファイル (config.yml) などの補助的なライブラリとして次.

- matplotlib: 3.9.2
- numpy: 2.3.3
- pandas: 2.3.0
- PyYAML: 6.0.2

C 公開レポジトリ

本研究で使用した実装は実験の再現性と透明性を高めるために, 次の GitHub リポジトリで公開している.

https://github.com/59GauthierLab/Katano_Research_Public

D 混同行列から得られる各モデルのクラス別指標

次ページに記す.

表 appx.1: テストデータに対する CNN モデルのクラス別指標

label_id	label_name	support	tp	fp	fn	precision	recall	f1
0	jpg	10182	9018	3101	1164	0.744121	0.885681	0.808753
1	arw	10302	10105	105	197	0.989716	0.980877	0.985277
2	cr2	10174	8996	1324	1178	0.871705	0.884215	0.877915
3	dng	10202	8629	4754	1573	0.644773	0.845815	0.731736
4	gpr	10166	10132	78	34	0.992360	0.996656	0.994503
5	nef	10321	9508	817	813	0.920872	0.921229	0.921050
6	nrw	10285	9945	319	340	0.968920	0.966942	0.967930
7	orf	10302	8966	2222	1336	0.801394	0.870316	0.834435
8	pef	10178	9787	962	391	0.910503	0.961584	0.935347
9	raf	10201	10093	62	108	0.993895	0.989413	0.991649
10	rw2	10339	9982	127	357	0.987437	0.965471	0.976330
11	3fr	10114	10091	32	23	0.996839	0.997726	0.997282
12	tiff	10165	9826	1184	339	0.892461	0.966650	0.928076
13	heic	10195	948	12001	9247	0.073210	0.092987	0.081922
14	bmp	10316	10095	1890	221	0.842303	0.978577	0.905341
15	gif	10302	9750	1485	552	0.867824	0.946418	0.905419
16	png	10056	7477	5303	2579	0.585055	0.743536	0.654843
17	ai	10192	2818	5504	7374	0.338621	0.276491	0.304418
18	eps	10194	10071	42	123	0.995847	0.987934	0.991875
19	psd	10305	9867	356	438	0.965177	0.957496	0.961321
20	mov	10246	1212	7275	9034	0.142807	0.118290	0.129397
21	mp4	10319	768	8811	9551	0.080175	0.074426	0.077194
22	3gp	10330	9193	2181	1137	0.808247	0.889932	0.847125
23	avi	10288	1336	2797	8952	0.323252	0.129860	0.185285
24	mkv	10136	9044	2153	1092	0.807716	0.892265	0.847888
25	ogv	10300	6815	4121	3485	0.623171	0.661650	0.641835
26	webm	10268	4244	6145	6024	0.408509	0.413323	0.410902
27	apk	10071	3693	6100	6378	0.377106	0.366696	0.371828
28	jar	10254	4480	5163	5774	0.464586	0.436903	0.450319
29	msi	10136	1242	3687	8894	0.251978	0.122534	0.164885
30	dmg	10391	1946	6633	8445	0.226833	0.187277	0.205166
31	7z	10318	736	9796	9582	0.069882	0.071332	0.070600
32	bz2	10350	1456	13552	8894	0.097015	0.140676	0.114836
33	deb	10384	1466	9793	8918	0.130207	0.141179	0.135471
34	gz	10320	1530	7347	8790	0.172356	0.148256	0.159400
35	pkg	10256	830	2346	9426	0.261335	0.080928	0.123585
36	rar	10265	2592	20200	7673	0.113724	0.252509	0.156820
37	rpm	10302	1305	1608	8997	0.447992	0.126674	0.197503
38	xz	10350	528	6958	9822	0.070532	0.051014	0.059206
39	zip	10297	1613	7349	8684	0.179982	0.156648	0.167506
40	exe	10030	603	7450	9427	0.074879	0.060120	0.066692
41	mach-o	10109	9482	208	627	0.978535	0.937976	0.957826
42	elf	10205	8949	262	1256	0.971556	0.876923	0.921817
43	dll	10261	9461	552	800	0.944872	0.922035	0.933314
44	doc	10265	9253	1105	1012	0.893319	0.901413	0.897348
45	docx	10204	1911	12182	8293	0.135599	0.187279	0.157303
46	key	10343	4796	2002	5547	0.705502	0.463695	0.559594
47	ppt	10198	4484	3754	5714	0.544307	0.439694	0.486440
48	pptx	10309	5131	2832	5178	0.644355	0.497720	0.561624
49	xls	10231	10168	19	63	0.998135	0.993842	0.995984
50	xlsx	10221	9942	551	279	0.947489	0.972703	0.959930
51	djvu	10224	1767	8087	8457	0.179318	0.172829	0.176014
52	epub	10237	3431	15735	6806	0.179015	0.335157	0.233378
53	mobi	10216	7411	44	2805	0.994098	0.725431	0.838775
54	pdf	10239	2494	1470	7745	0.629162	0.243578	0.351193
55	md	10195	9998	203	197	0.980100	0.980677	0.980388
56	rtf	10326	10288	146	38	0.986007	0.996320	0.991137
57	txt	10199	9678	374	521	0.962793	0.948917	0.955805
58	tex	10225	9897	227	328	0.977578	0.967922	0.972726
59	json	10215	10193	24	22	0.997651	0.997846	0.997749
60	html	10220	10016	103	204	0.989821	0.980039	0.984906
61	xml	10263	10262	0	1	1.000000	0.999903	0.999951
62	log	10073	10069	0	4	1.000000	0.999603	0.999801
63	csv	10059	10030	14	29	0.998606	0.997117	0.997861
64	aiff	10283	10242	55	41	0.994659	0.996013	0.995335
65	flac	10176	7025	15848	3151	0.307131	0.690350	0.425126
66	m4a	10318	9852	780	466	0.926637	0.954836	0.940525
67	mp3	10304	9864	608	440	0.941940	0.957298	0.949557
68	ogg	10375	9541	2284	834	0.806850	0.919614	0.859550
69	wav	10145	10145	6	0	0.999409	1.000000	0.999704
70	wma	10287	10153	227	134	0.978131	0.986974	0.982533
71	pcap	10126	5071	550	5055	0.902153	0.500790	0.644059
72	ttf	10278	10141	130	137	0.987343	0.986671	0.987007
73	dwg	10281	9921	414	360	0.959942	0.964984	0.962456
74	sqlite	10288	10168	101	120	0.990165	0.988336	0.989249

表 appx.2: テストデータに対する GRU モデルのクラス別指標

label_id	label_name	support	tp	fp	fn	precision	recall	f1
0	jpg	10182	9311	1349	871	0.873452	0.914457	0.893484
1	arw	10302	10102	114	200	0.988841	0.980586	0.984696
2	cr2	10174	9511	765	663	0.925555	0.934834	0.930171
3	dng	10202	9005	2777	1197	0.764301	0.882670	0.819232
4	gpr	10166	10135	30	31	0.997049	0.996951	0.997000
5	nef	10321	9727	559	594	0.945654	0.942447	0.944048
6	nrw	10285	10003	268	282	0.973907	0.972581	0.973244
7	orf	10302	9379	1208	923	0.885898	0.910406	0.897985
8	pef	10178	9879	574	299	0.945088	0.970623	0.957685
9	raf	10201	10145	27	56	0.997346	0.994510	0.995926
10	rw2	10339	10066	155	273	0.984835	0.973595	0.979183
11	3fr	10114	10108	25	6	0.997533	0.999407	0.998469
12	tiff	10165	9803	569	362	0.945141	0.964388	0.954667
13	heic	10195	1093	12842	9102	0.078436	0.107209	0.090593
14	bmp	10316	10107	1752	209	0.852264	0.979740	0.911567
15	gif	10302	9660	1242	642	0.886076	0.937682	0.911149
16	png	10056	8115	3140	1941	0.721013	0.806981	0.761579
17	ai	10192	3501	5987	6691	0.368992	0.343505	0.355793
18	eps	10194	10068	31	126	0.996930	0.987640	0.992263
19	psd	10305	10015	262	290	0.974506	0.971858	0.973180
20	mov	10246	1492	7473	8754	0.166425	0.145618	0.155328
21	mp4	10319	969	8155	9350	0.106203	0.093904	0.099676
22	3gp	10330	9696	938	634	0.911792	0.938625	0.925014
23	avi	10288	1330	1378	8958	0.491137	0.129277	0.204678
24	mkv	10136	9367	832	769	0.918423	0.924132	0.921269
25	ogv	10300	8212	1714	2088	0.827322	0.797282	0.812024
26	webm	10268	4373	3143	5895	0.581825	0.425886	0.491790
27	apk	10071	4580	4514	5491	0.503629	0.454771	0.477955
28	jar	10254	5450	3182	4804	0.631372	0.531500	0.577147
29	msi	10136	1785	2906	8351	0.380516	0.176105	0.240777
30	dmg	10391	2550	5029	7841	0.336456	0.245405	0.283806
31	7z	10318	858	10528	9460	0.075356	0.083156	0.079064
32	bz2	10350	3666	32630	6684	0.101003	0.354203	0.157184
33	deb	10384	1860	6794	8524	0.214930	0.179122	0.195399
34	gz	10320	2494	6430	7826	0.279471	0.241667	0.259198
35	pkg	10256	1375	3967	8881	0.257394	0.134068	0.176305
36	rar	10265	3076	18666	7189	0.141477	0.299659	0.192208
37	rpm	10302	1540	1512	8762	0.504587	0.149486	0.230643
38	xz	10350	514	6424	9836	0.074085	0.049662	0.059463
39	zip	10297	2495	4628	7802	0.350274	0.242304	0.286452
40	exe	10030	443	5151	9587	0.079192	0.044167	0.056708
41	mach-o	10109	9576	190	533	0.980545	0.947275	0.963623
42	elf	10205	9050	222	1155	0.976057	0.886820	0.929301
43	dll	10261	9615	397	646	0.960348	0.937043	0.948552
44	doc	10265	9699	563	566	0.945137	0.944861	0.944999
45	docx	10204	2093	9606	8111	0.178904	0.205116	0.191115
46	key	10343	5802	1890	4541	0.754290	0.560959	0.643416
47	ppt	10198	5727	3377	4471	0.629064	0.561581	0.593410
48	pptx	10309	6022	2399	4287	0.715117	0.584150	0.643033
49	xls	10231	10172	18	59	0.998234	0.994233	0.996229
50	xlsx	10221	9916	392	305	0.961971	0.970159	0.966048
51	djvu	10224	2026	6847	8198	0.228333	0.198161	0.212180
52	epub	10237	4166	13706	6071	0.233102	0.406955	0.296418
53	mobi	10216	7436	56	2780	0.992525	0.727878	0.839846
54	pdf	10239	3082	1535	7157	0.667533	0.301006	0.414917
55	md	10195	10125	123	70	0.987998	0.993134	0.990559
56	rtf	10326	10312	77	14	0.992588	0.998644	0.995607
57	txt	10199	9695	133	504	0.986467	0.950583	0.968193
58	tex	10225	10047	199	178	0.980578	0.982592	0.981584
59	json	10215	10199	2	16	0.999804	0.998434	0.999118
60	html	10220	10094	87	126	0.991455	0.987671	0.989559
61	xml	10263	10260	0	3	1.000000	0.999708	0.999854
62	log	10073	10068	3	5	0.999702	0.999504	0.999603
63	csv	10059	10042	18	17	0.998211	0.998310	0.998260
64	aiff	10283	10251	52	32	0.994953	0.996888	0.995920
65	flac	10176	7758	14954	2418	0.341582	0.762382	0.471783
66	m4a	10318	10190	48	128	0.995312	0.987594	0.991438
67	mp3	10304	10081	247	223	0.976084	0.978358	0.977220
68	ogg	10375	9912	654	463	0.938103	0.955373	0.946660
69	wav	10145	10141	7	4	0.999310	0.999606	0.999458
70	wma	10287	10208	34	79	0.996680	0.992320	0.994496
71	pcap	10126	5977	2045	4149	0.745076	0.590263	0.658695
72	ttf	10278	10185	112	93	0.989123	0.990952	0.990036
73	dwg	10281	9983	274	298	0.973287	0.971014	0.972149
74	sqlite	10288	10262	33	26	0.996795	0.997473	0.997134

表 appx.3: テストデータに対する LSTM モデルのクラス別指標

label_id	label_name	support	tp	fp	fn	precision	recall	f1
0	jpg	10182	9124	2738	1058	0.769179	0.896091	0.827799
1	arw	10302	10067	97	235	0.990457	0.977189	0.983778
2	cr2	10174	9213	943	961	0.907148	0.905544	0.906345
3	dng	10202	8936	4543	1266	0.662957	0.875907	0.754698
4	gpr	10166	10111	60	55	0.994101	0.994590	0.994345
5	nef	10321	9396	919	925	0.910906	0.910377	0.910642
6	nrw	10285	9921	313	364	0.969416	0.964609	0.967006
7	orf	10302	9097	1820	1205	0.833288	0.883032	0.857439
8	pef	10178	9691	787	487	0.924890	0.952152	0.938323
9	raf	10201	10082	60	119	0.994084	0.988334	0.991201
10	rw2	10339	9928	89	411	0.991115	0.960248	0.975437
11	3fr	10114	10083	32	31	0.996836	0.996935	0.996886
12	tiff	10165	9749	1134	416	0.895801	0.959075	0.926359
13	heic	10195	1274	14484	8921	0.080848	0.124963	0.098177
14	bmp	10316	10099	1807	217	0.848228	0.978965	0.908919
15	gif	10302	9498	1984	804	0.827208	0.921957	0.872016
16	png	10056	7598	5137	2458	0.596623	0.755569	0.666754
17	ai	10192	2664	5553	7528	0.324206	0.261381	0.289424
18	eps	10194	10062	47	132	0.995351	0.987051	0.991184
19	psd	10305	9823	359	482	0.964742	0.953227	0.958950
20	mov	10246	1657	8857	8589	0.157599	0.161722	0.159634
21	mp4	10319	828	7726	9491	0.096797	0.080240	0.087744
22	3gp	10330	9359	1486	971	0.862978	0.906002	0.883967
23	avi	10288	1259	1021	9029	0.552193	0.122376	0.200350
24	mkv	10136	9166	1389	970	0.868404	0.904301	0.885989
25	ogv	10300	7409	3603	2891	0.672811	0.719320	0.695289
26	webm	10268	4327	3340	5941	0.564367	0.421406	0.482520
27	apk	10071	3982	5913	6089	0.402425	0.395393	0.398878
28	jar	10254	5340	5940	4914	0.473404	0.520772	0.495960
29	msi	10136	1029	3407	9107	0.231966	0.101519	0.141230
30	dmg	10391	2397	7395	7994	0.244792	0.230680	0.237527
31	7z	10318	760	9607	9558	0.073310	0.073658	0.073483
32	bz2	10350	2795	27241	7555	0.093055	0.270048	0.138414
33	deb	10384	1517	8904	8867	0.145571	0.146090	0.145830
34	gz	10320	1817	6350	8503	0.222481	0.176066	0.196571
35	pkg	10256	770	2847	9486	0.212884	0.075078	0.111007
36	rar	10265	3013	22257	7252	0.119232	0.293522	0.169579
37	rpm	10302	1451	2181	8851	0.399504	0.140846	0.208268
38	xz	10350	443	5572	9907	0.073649	0.042802	0.054140
39	zip	10297	1741	4966	8556	0.259580	0.169078	0.204775
40	exe	10030	301	3894	9729	0.071752	0.030010	0.042320
41	mach-o	10109	9476	221	633	0.977209	0.937383	0.956882
42	elf	10205	8943	304	1262	0.967124	0.876335	0.919494
43	dll	10261	9514	631	747	0.937802	0.927200	0.932471
44	doc	10265	9384	1044	881	0.899885	0.914174	0.906973
45	docx	10204	1433	8596	8771	0.142886	0.140435	0.141650
46	key	10343	5030	1555	5313	0.763857	0.486319	0.594282
47	ppt	10198	4857	3632	5341	0.572152	0.476270	0.519827
48	pptx	10309	5438	2549	4871	0.680856	0.527500	0.594447
49	xls	10231	10173	14	58	0.998626	0.994331	0.996474
50	xlsx	10221	9820	532	401	0.948609	0.960767	0.954649
51	djvu	10224	1933	6182	8291	0.238201	0.189065	0.210808
52	epub	10237	3863	15756	6374	0.196901	0.377357	0.258775
53	mobi	10216	7402	48	2814	0.993557	0.724550	0.837994
54	pdf	10239	2464	766	7775	0.762848	0.240649	0.365877
55	md	10195	10118	158	77	0.984624	0.992447	0.988520
56	rtf	10326	10320	105	6	0.989928	0.999419	0.994651
57	txt	10199	9720	184	479	0.981422	0.953035	0.967020
58	tex	10225	9984	211	241	0.979304	0.976430	0.977865
59	json	10215	10190	9	25	0.999118	0.997553	0.998334
60	html	10220	10061	100	159	0.990158	0.984442	0.987292
61	xml	10263	10255	2	8	0.999805	0.999221	0.999513
62	log	10073	10067	5	6	0.999504	0.999404	0.999454
63	csv	10059	10025	30	34	0.997016	0.996620	0.996818
64	aiff	10283	10222	64	61	0.993778	0.994068	0.993923
65	flac	10176	7178	14593	2998	0.329705	0.705385	0.449369
66	m4a	10318	10153	108	165	0.989475	0.984009	0.986734
67	mp3	10304	9988	236	316	0.976917	0.969332	0.973110
68	ogg	10375	9695	1311	680	0.880883	0.934458	0.906880
69	wav	10145	10141	6	4	0.999409	0.999606	0.999507
70	wma	10287	10182	71	105	0.993075	0.989793	0.991431
71	pcap	10126	5249	211	4877	0.961355	0.518369	0.673553
72	ttf	10278	10162	168	116	0.983737	0.988714	0.986219
73	dwg	10281	9961	335	320	0.967463	0.968875	0.968168
74	sqlite	10288	10236	47	52	0.995429	0.994946	0.995187

表 appx.4: テストデータに対する Transformer モデルのクラス別指標

label_id	label_name	support	tp	fp	fn	precision	recall	f1
0	jpg	10182	9222	3008	960	0.754047	0.905716	0.822952
1	arw	10302	10098	282	204	0.972832	0.980198	0.976501
2	cr2	10174	9378	776	796	0.923577	0.921761	0.922668
3	dng	10202	9122	4249	1080	0.682223	0.894138	0.773936
4	gpr	10166	10110	61	56	0.994003	0.994491	0.994247
5	nef	10321	9565	656	756	0.935818	0.926751	0.931263
6	nrw	10285	9942	211	343	0.979218	0.966650	0.972894
7	orf	10302	9047	1758	1255	0.837298	0.878179	0.857251
8	pef	10178	9941	2760	237	0.782694	0.976714	0.869007
9	raf	10201	10102	41	99	0.995958	0.990295	0.993118
10	rw2	10339	9919	50	420	0.994984	0.959377	0.976856
11	3fr	10114	10084	38	30	0.996246	0.997034	0.996640
12	tiff	10165	9794	4691	371	0.676148	0.963502	0.794645
13	heic	10195	762	9947	9433	0.071155	0.074743	0.072905
14	bmp	10316	10099	1793	217	0.849226	0.978965	0.909492
15	gif	10302	9984	225	318	0.977961	0.969132	0.973526
16	png	10056	7425	4865	2631	0.604150	0.738365	0.664548
17	ai	10192	2703	2399	7489	0.529792	0.265208	0.353472
18	eps	10194	10070	30	124	0.997030	0.987836	0.992412
19	psd	10305	9954	577	351	0.945209	0.965939	0.955462
20	mov	10246	1948	13501	8298	0.126092	0.190123	0.151625
21	mp4	10319	930	9294	9389	0.090962	0.090125	0.090542
22	3gp	10330	9583	1278	747	0.882331	0.927686	0.904441
23	avi	10288	1263	598	9025	0.678667	0.122764	0.207918
24	mkv	10136	9522	2334	614	0.803138	0.939424	0.865951
25	ogv	10300	6990	775	3310	0.900193	0.678641	0.773872
26	webm	10268	4370	4000	5898	0.522103	0.425594	0.468934
27	apk	10071	3913	3440	6158	0.532164	0.388541	0.449151
28	jar	10254	4126	1276	6128	0.763791	0.402380	0.527082
29	msi	10136	1057	1125	9079	0.484418	0.104282	0.171619
30	dmg	10391	1422	1536	8969	0.480730	0.136849	0.213050
31	7z	10318	381	5869	9937	0.060960	0.036926	0.045992
32	bz2	10350	5879	68105	4471	0.079463	0.568019	0.139422
33	deb	10384	866	2334	9518	0.270625	0.083398	0.127503
34	gz	10320	1792	4227	8528	0.297724	0.173643	0.219352
35	pkg	10256	719	1140	9537	0.386767	0.070105	0.118696
36	rar	10265	1462	7944	8803	0.155433	0.142426	0.148645
37	rpm	10302	1568	1823	8734	0.462400	0.152203	0.229022
38	xz	10350	69	1311	10281	0.050000	0.006667	0.011765
39	zip	10297	2071	5600	8226	0.269978	0.201127	0.230521
40	exe	10030	345	4075	9685	0.078054	0.034397	0.047751
41	mach-o	10109	9590	314	519	0.968296	0.948660	0.958377
42	elf	10205	9015	309	1190	0.966860	0.883390	0.923242
43	dll	10261	9698	2025	563	0.827263	0.945132	0.882278
44	doc	10265	9406	975	859	0.906078	0.916318	0.911169
45	docx	10204	957	3148	9247	0.233130	0.093787	0.133762
46	key	10343	4568	2286	5775	0.666472	0.441651	0.531255
47	ppt	10198	5448	3735	4750	0.593270	0.534222	0.562200
48	pptx	10309	5533	3816	4776	0.591828	0.536715	0.562926
49	xls	10231	10178	31	53	0.996963	0.994820	0.995890
50	xlsx	10221	9718	267	503	0.973260	0.950788	0.961893
51	djvu	10224	1912	6268	8312	0.233741	0.187011	0.207781
52	epub	10237	4333	16999	5904	0.203122	0.423269	0.274510
53	mobi	10216	7429	39	2787	0.994778	0.727193	0.840195
54	pdf	10239	2941	1732	7298	0.629360	0.287235	0.394447
55	md	10195	9971	114	224	0.988696	0.978028	0.983333
56	rtf	10326	10319	91	7	0.991258	0.999322	0.995274
57	txt	10199	9245	319	954	0.966646	0.906461	0.935587
58	tex	10225	10066	674	159	0.937244	0.984450	0.960267
59	json	10215	10203	7	12	0.999314	0.998825	0.999070
60	html	10220	10068	102	152	0.989971	0.985127	0.987543
61	xml	10263	10263	9	0	0.999124	1.000000	0.999562
62	log	10073	10070	3	3	0.999702	0.999702	0.999702
63	csv	10059	10029	8	30	0.999203	0.997018	0.998109
64	aiff	10283	10113	95	170	0.990694	0.983468	0.987067
65	flac	10176	7780	19083	2396	0.289618	0.764544	0.420098
66	m4a	10318	10210	497	108	0.953582	0.989533	0.971225
67	mp3	10304	10049	455	255	0.956683	0.975252	0.965879
68	ogg	10375	9688	1091	687	0.898785	0.933783	0.915950
69	wav	10145	10143	7	2	0.999310	0.999803	0.999557
70	wma	10287	10198	14	89	0.998629	0.991348	0.994975
71	pcap	10126	5719	481	4407	0.922419	0.564784	0.700600
72	ttf	10278	10158	93	120	0.990928	0.988325	0.989624
73	dwg	10281	9954	118	327	0.988284	0.968194	0.978136
74	sqlite	10288	10197	27	91	0.997359	0.991155	0.994247

E 混同行列を図示したグラフ

このグラフは、縦軸が真のクラス、横軸が予測クラス、色の濃さ（Count）が件数（サンプル数）を表す。対角成分が大きいほど正分類件数が多く、非対角成分はクラス間の誤分類傾向（取り間違えた件数）を示す。「全て間違わない場合」の理論上の見え方は、非対角成分がすべて0で、対角成分 (i, i) が各クラスのサンプル数 N_i になる。このとき1セルの理論上の最大値は $\max_i N_i$ （評価データ中で最も件数の多いクラスのサンプル数）である。

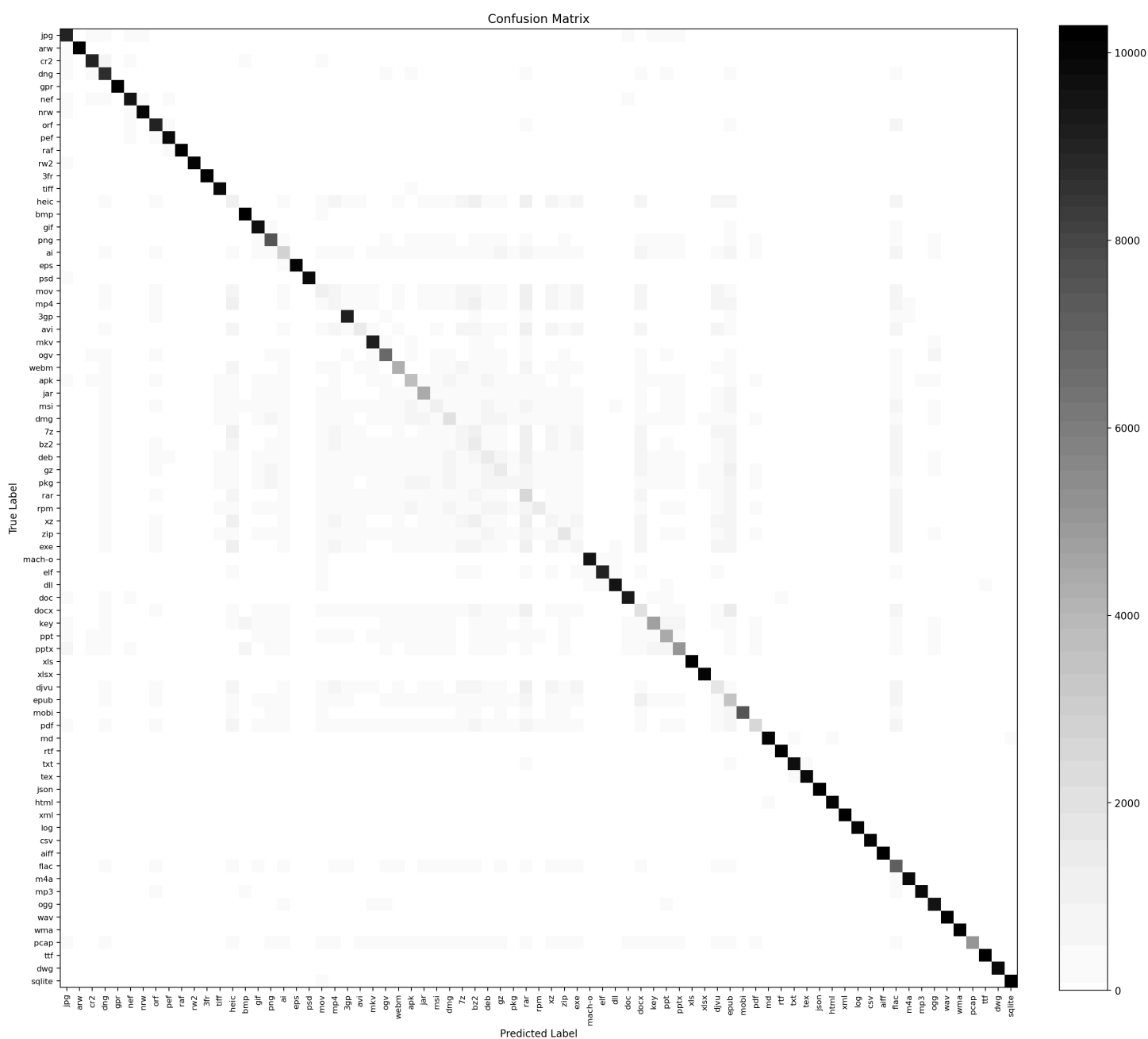


図 appx.1: CNN の結果に対する混同行列

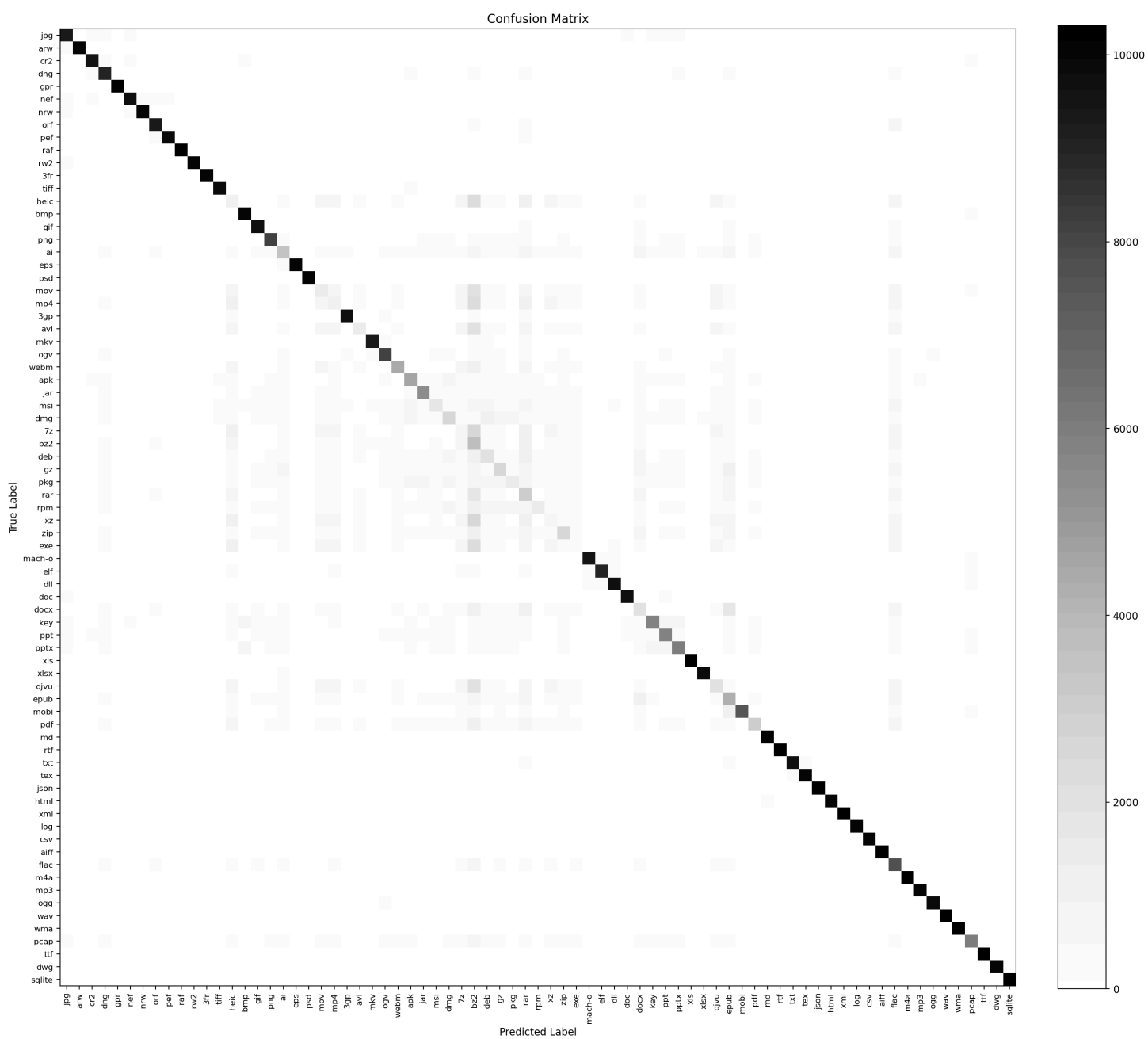


図 appx.2: GRU の結果に対する混同行列

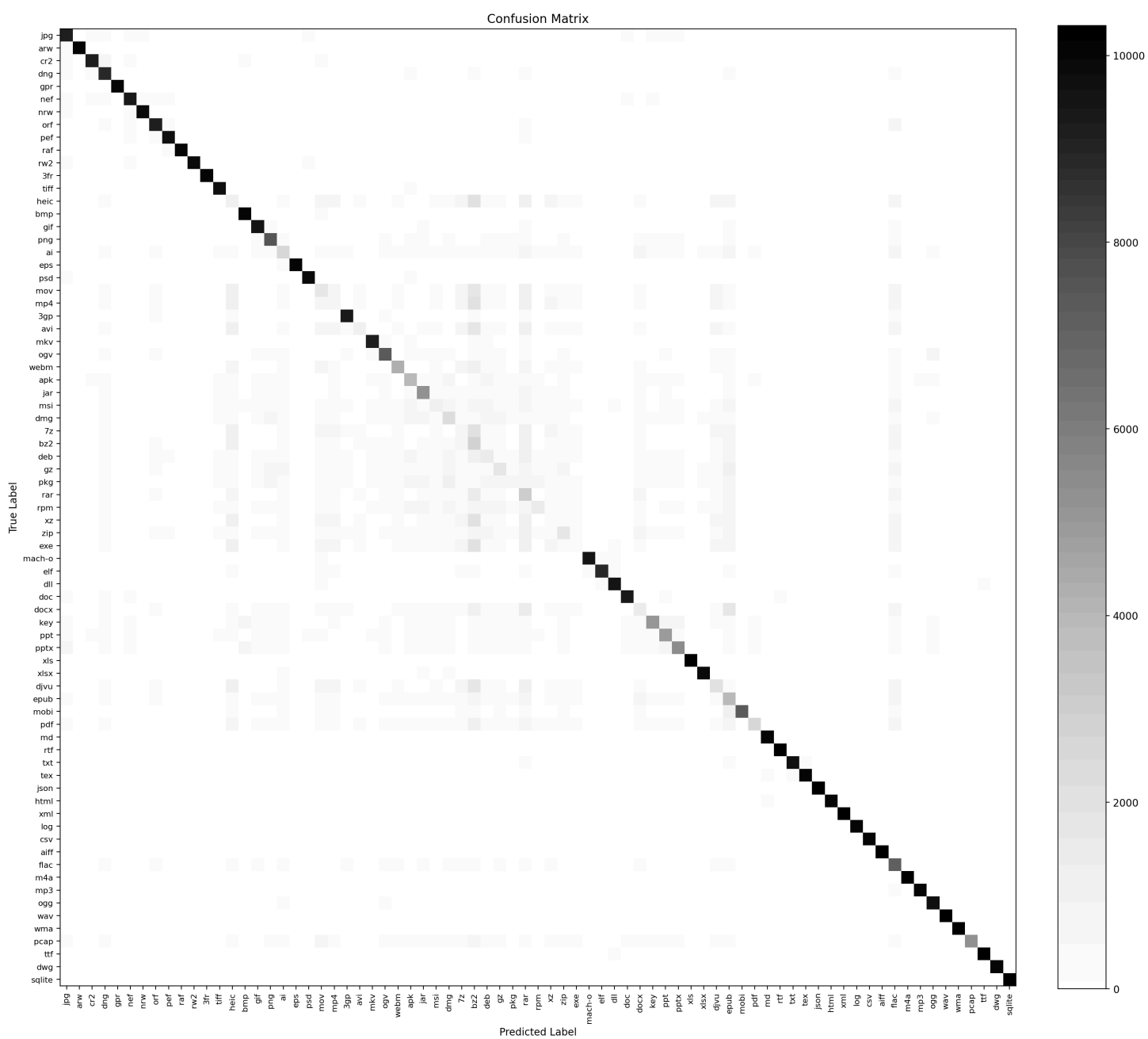


図 appx.3: LSTM の結果に対する混同行列

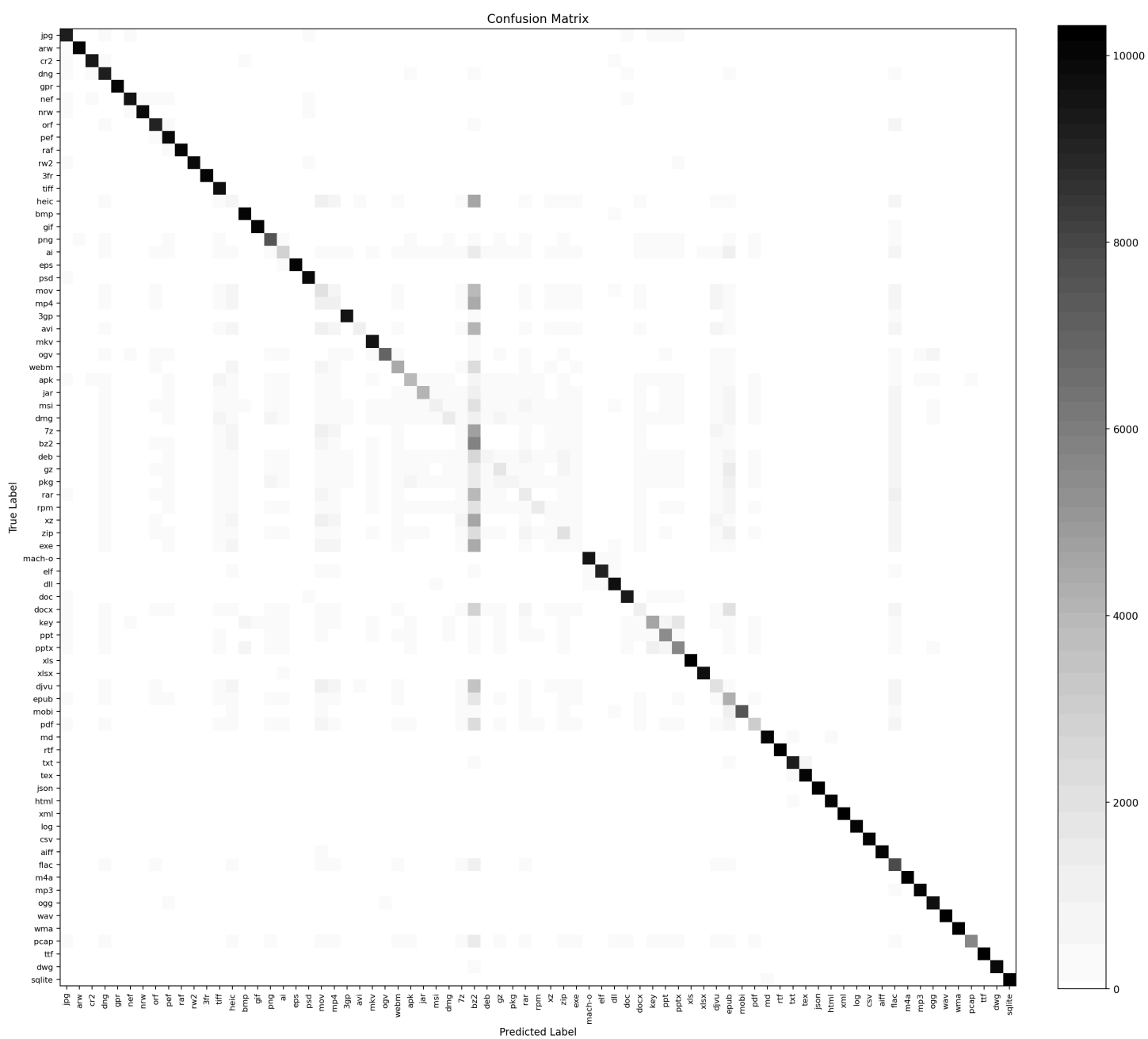


図 appx.4: Transformer の結果に対する混同行列